



Image Classification with Ensemble Stacking Techniques

Eric Cao
Shauna Song



Goal

Improve image classification accuracy

- Combine strengths of different Network
- Minimize loss and reduce overfitting
- Enhance the stability of the image classification system
- Demonstrate better performance of ensemble stacking compared to individual models



Method

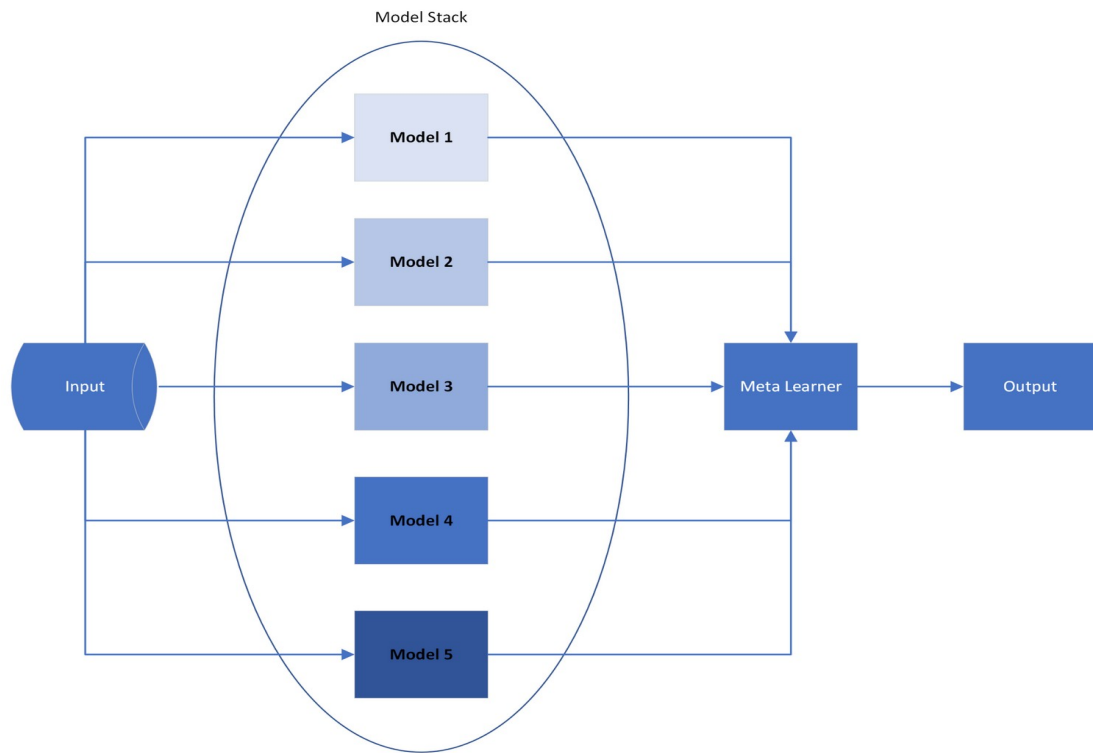
Image classification - CNN(eg. LeNet, AlexNet, ResNet, VGG16)

Ensemble learning - stacking

- Combines multiple neural network models to improve overall performance
- Reduces both bias and variance for better classification results

Drawbacks:

- Task-dependent: The effectiveness of ensemble learning depends on the specific task and the choice of base models.
- At risk of overfitting: Ensemble stacking, if not properly managed, can lead to overfitting.

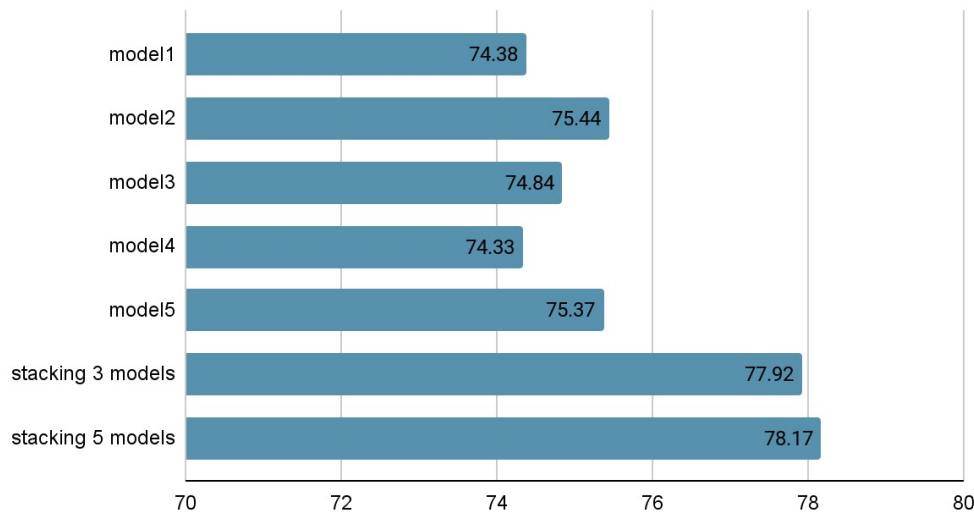


Data set - CIFAR-10

- Widely-used benchmark for image classification tasks
- Contains 60,000 color images of size 32x32 pixels
- Small image size results in lower computational requirements
- 10 distinct classes: airplanes, automobiles, birds, cats, deer, dogs, frogs, horses, ships, and trucks
- Balanced distribution: 6,000 images per class (5,000 for training, 1,000 for testing)

Experiment -LeNet

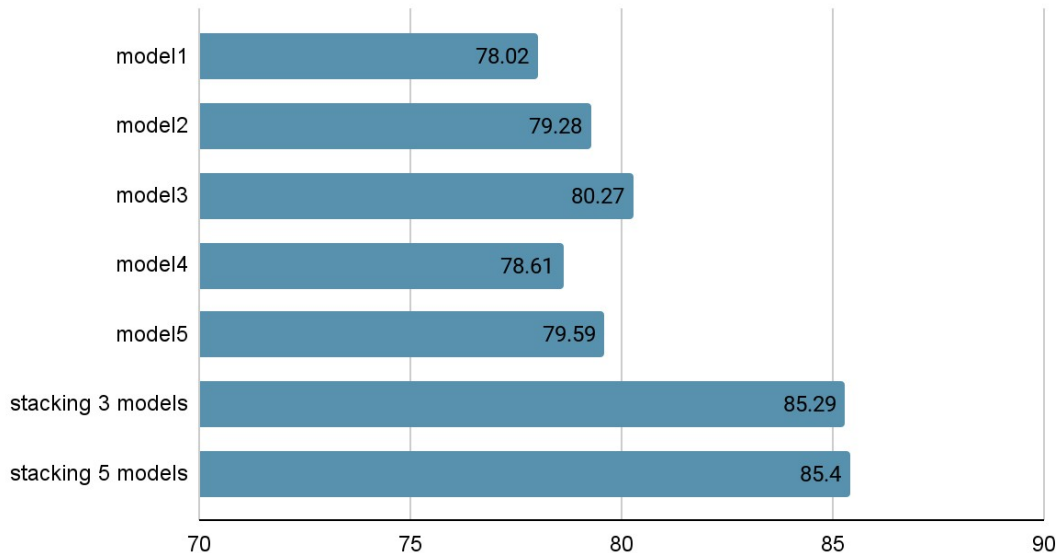
LeNet



```
LeNet(  
    (conv1): Conv2d(3, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (conv2): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (conv3): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (fc1): Linear(in_features=1024, out_features=500, bias=True)  
    (dropout1): Dropout(p=0.5, inplace=False)  
    (fc2): Linear(in_features=500, out_features=10, bias=True)  
)
```

Experiment- Alexnet

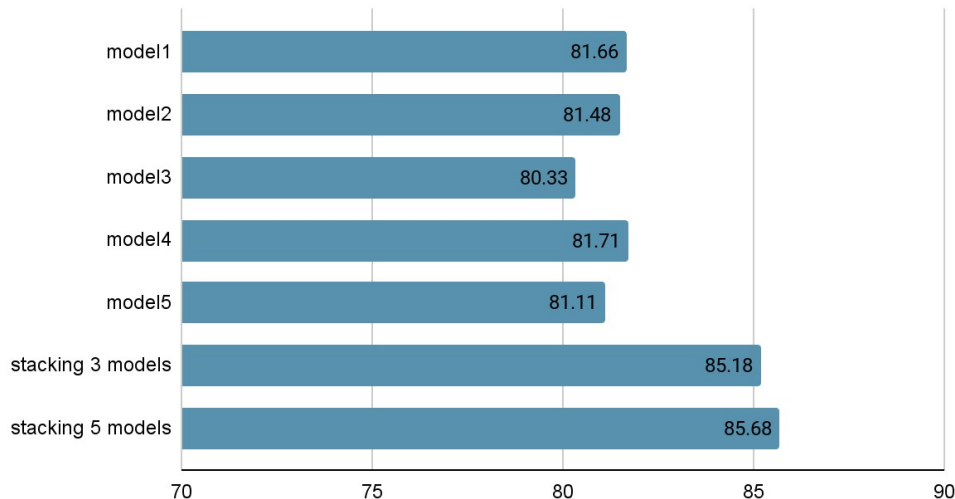
AlexNet



```
AlexNet(  
  (features): Sequential(  
    (0): Conv2d(3, 64, kernel_size=(11, 11), stride=(4, 4), padding=(2, 2))  
    (1): ReLU(inplace=True)  
    (2): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)  
    (3): Conv2d(64, 192, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))  
    (4): ReLU(inplace=True)  
    (5): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)  
    (6): Conv2d(192, 384, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (7): ReLU(inplace=True)  
    (8): Conv2d(384, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (9): ReLU(inplace=True)  
    (10): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (11): ReLU(inplace=True)  
    (12): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)  
  )  
  (avgpool): AdaptiveAvgPool2d(output_size=(6, 6))  
  (classifier): Sequential(  
    (0): Dropout(p=0.5, inplace=False)  
    (1): Linear(in_features=9216, out_features=4096, bias=True)  
    (2): ReLU(inplace=True)  
    (3): Dropout(p=0.5, inplace=False)  
    (4): Linear(in_features=4096, out_features=4096, bias=True)  
    (5): ReLU(inplace=True)  
    (6): Linear(in_features=4096, out_features=1000, bias=True)  
  )  
)
```

Experiment-Resnet

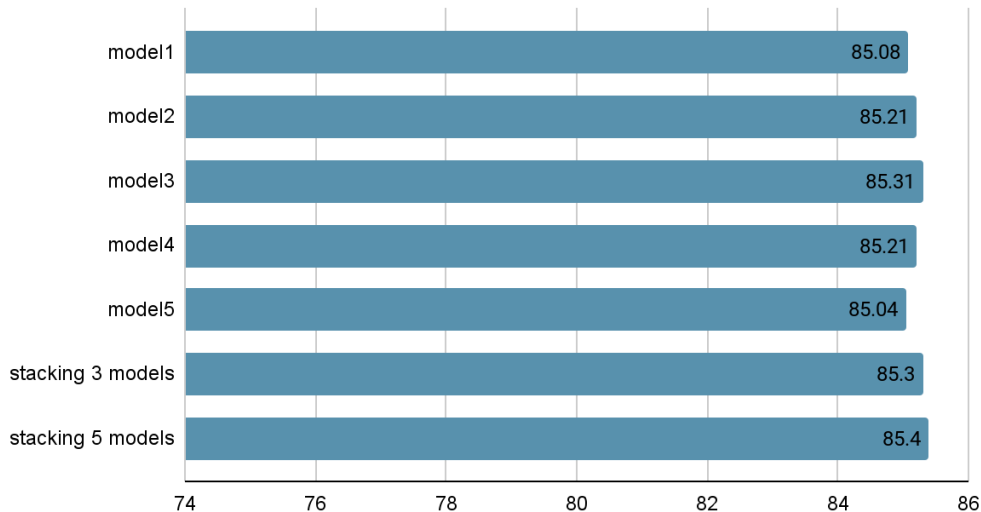
ResNet



```
ResNetModel(  
  (resnet): ResNet(  
    (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)  
    (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (relu): ReLU(inplace=True)  
    (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)  
    (layer1): Sequential(  
      (0): BasicBlock(  
        (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
        (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
        (relu): ReLU(inplace=True)  
        (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
        (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      )  
      (1): BasicBlock(  
        (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
        (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
        (relu): ReLU(inplace=True)  
        (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
        (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      )  
    )  
    (layer2): Sequential(  
      (0): BasicBlock(  
        (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)  
        .....  
      )  
    (layer3): Sequential(  
      (0): BasicBlock(  
        (conv1): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)  
        .....  
      )  
    (layer4): Sequential(  
      (0): BasicBlock(  
        (conv1): Conv2d(256, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)  
        .....  
      )  
    (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))  
    (fc): Linear(in_features=512, out_features=10, bias=True)  
  )  
)
```

Experiment -VGG16

VGG16



```
VGG(  
  (features): Sequential(  
    (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (1): ReLU(inplace=True)  
    (2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (3): ReLU(inplace=True)  
    (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
    (5): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (6): ReLU(inplace=True)  
    (7): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (8): ReLU(inplace=True)  
    (9): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
    (10): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (11): ReLU(inplace=True)  
    (12): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (13): ReLU(inplace=True)  
    (14): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (15): ReLU(inplace=True)  
    (16): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
    (17): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (18): ReLU(inplace=True)  
    (19): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (20): ReLU(inplace=True)  
    (21): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (22): ReLU(inplace=True)  
    (23): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
    (24): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (25): ReLU(inplace=True)  
    (26): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (27): ReLU(inplace=True)  
    (28): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (29): ReLU(inplace=True)  
    (30): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
  )  
  (avgpool): AdaptiveAvgPool2d(output_size=(7, 7))  
  (classifier): Sequential(  
    (0): Linear(in_features=25088, out_features=4096, bias=True)  
    (1): ReLU(inplace=True)  
    (2): Dropout(p=0.5, inplace=False)  
    (3): Linear(in_features=4096, out_features=4096, bias=True)  
    (4): Linear(in_features=4096, out_features=1024, bias=True)  
    (5): Dropout(p=0.5, inplace=False)  
    (6): Linear(in_features=1024, out_features=10, bias=True)  
  )  
)
```

Experiment-Mixed network

LeNet 74.53

AlexNet 79.95

ResNet 81.41

VGG16 86.36

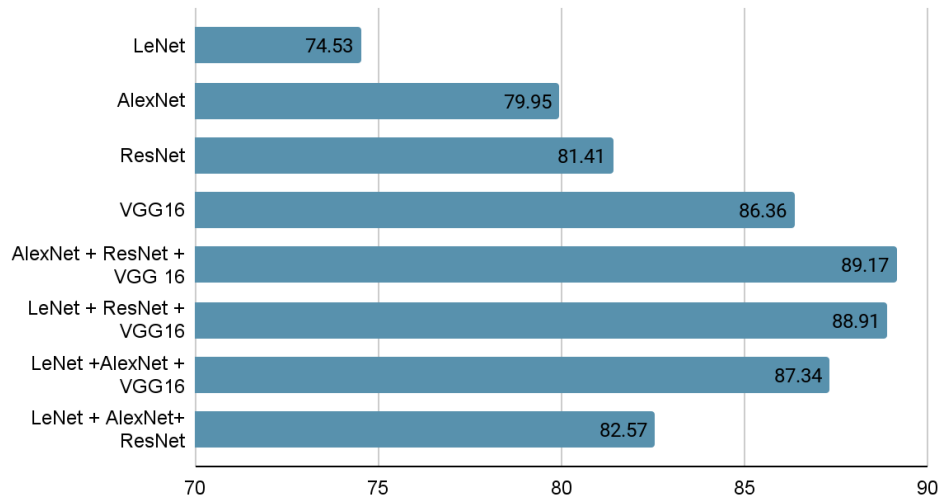
AlexNet + ResNet + VGG16 89.17

LeNet + AlexNet + VGG16 88.91

LeNet + ResNet + VGG16 87.34

LeNet + AlexNet + ResNet 82.57

Mixed Network





Conclusion

- Stacking boosts performance but requires more computing power.
- Stacking more models yield better results, but with small improvements.
- Mixed network ensembles yield better performance improvements, but at higher computational costs.



Future direction

- Evaluate ensemble learning improvement using Vision Transformer
- Utilize model soups, which improve accuracy by averaging the weights of multiple models' parameters without increasing inference time